

VDR-LLM-Prolog: The Compound Architecture Performance Gains

Exact Integer Arithmetic as Foundation for Complete LLM System Redesign

Registry: [\[HOWL-VDR-33-2026\]](#)

Series Path: [\[HOWL-VDR-1-2026\]](#) → [\[HOWL-VDR-2-2026\]](#) → [\[HOWL-MATH-3-2026\]](#) → [\[HOWL-MATH-4-2026\]](#) → ... → [\[HOWL-VDR-14-2026\]](#) → ... → [\[HOWL-VDR-21-2026\]](#) → [\[HOWL-VDR-22-2026\]](#) → [\[HOWL-VDR-23-2026\]](#) → [\[HOWL-VDR-24-2026\]](#) → [\[HOWL-VDR-25-2026\]](#) → [\[HOWL-VDR-26-2026\]](#) → [\[HOWL-VDR-27-2026\]](#) → [\[HOWL-VDR-28-2026\]](#) → [\[HOWL-VDR-29-2026\]](#) → [\[HOWL-VDR-30-2026\]](#) → [\[HOWL-VDR-31-2026\]](#) → [\[HOWL-VDR-32-2026\]](#) → [\[HOWL-VDR-33-2026\]](#)

DOI: 10.5281/zenodo.20284066

Date: May 2026

Domain: ML Infrastructure Economics / Exact Arithmetic

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Opus 4.6.

Abstract

Thirty-two papers in the VDR series each prove an independent result: exact arithmetic with zero error, instruction-level equivalence with quantized inference, 85-97% token elimination for structured tasks, linear scaling versus quadratic, self-improving rule accumulation, zero-drift diffusion chains, structural safety without token cost, and grammar-directed generation that eliminates forward passes on deterministic tokens. Each paper is conservative, staying within its own scope. None multiplies the results together.

This paper performs that multiplication. The axes of improvement are independent — hardware speedup does not depend on token reduction, token reduction does not depend on rule accumulation, rule accumulation does not depend on scaling behavior. When independent multipliers compound across a real workload over a real deployment timeline, the combined effect ranges from $2 \times$ for pure creative writing to over $8,000 \times$ for mature structured enterprise workloads. These are not projections from novel research. They are arithmetic consequences of measured baselines and known operations on shipping hardware.

1. The Foundation: Exact Integer Arithmetic

Every number in the VDR system is an ordered triple $[V, D, R]$ where V is an integer value, D is a nonzero integer denominator, and R is a remainder. The remainder is not error, not residue, not rounding noise. It is the exact structure that the denominator frame could not absorb, preserved as a first-class component of the value.

When D is a power of two, the divmod operation that separates V from R is a bit shift and a mask — the cheapest operations on any processor. Fix D at 2^{16} for inference weights, 2^{32} for schedule constants, 2^{64} for gradient accumulation. The denominator never changes through any operation. Overflow goes to R via divmod, not to wider denominators. The arithmetic is exact at every step, at every chain length, on every platform.

This is validated across 921 tests in 38 mathematical and computational domains — number theory, polynomial algebra, continued fractions, matrix decomposition, combinatorics, signal processing, computational geometry, differential equations, probability, cryptography, symbolic algebra, graph theory, game theory, coding theory, algebraic topology, tropical algebra, control theory, wavelets, chaos theory, transcendental arithmetic, and 14 physics domains including QED, quantum mechanics, orbital mechanics, and optics. Zero VDR computation errors. All 18 test failures across the series trace to test-design errors, never to incorrect arithmetic. The system remains falsifiable: any test producing an incorrect exact rational from correct inputs would falsify VDR. 921 tests have not produced one.

The Q335 basis ($D = 2^{335}$, providing 100 decimal digits of precision) exists to prove universality. It handles transcendental constants, elliptic integrals, higher zeta values, QED coefficients — every mathematical domain without exception. It is not the production configuration. Production inference uses Q8 weights and Q16 activations, matching hardware register widths, where the instruction sequence is identical to INT8/INT16 quantized inference. Q335 proves the arithmetic works everywhere. Q16 proves it runs at hardware speed. They are the same arithmetic at different denominator settings.

The Python reference implementation shipped as `vdr-math 0.1.0` on PyPI on May 19, 2026. MIT license, pure Python, no external dependencies, 151.8 KB wheel. The Zig toy implementation measures 688 ns per forward pass on a 2019 laptop — 1.42 million tokens per second, 2,368 bytes total memory, zero heap allocations, zero floating-point operations. The per-parameter cost of 3.80 ns scales linearly to SIMD and tensor core projections because the instruction sequence does not change with model size.

2. Why 80-95% of LLM Compute Is Wasted

A language model runs a full forward pass through every transformer layer, computes attention across the entire context, and evaluates softmax over 50,000+ vocabulary tokens to produce each output token. This costs the same whether the token carries 15 bits of genuine creative entropy or 0.1 bits of syntactically determined structure.

When an LLM generates a JSON response, the majority of forward passes produce braces,

brackets, colons, commas, quotation marks, and field names. These tokens were determined the moment the output format was decided. The model is spending billions of floating-point operations per token to arrive at characters that are the only legal option given the output structure.

When an LLM writes Zig source code, 70-80% of tokens are structural — `pub`, `const`, `fn`, `struct`, `{`, `}`, `(`, `)`, `:`, `;`, `=`, `return`. The actual information content is names, types, and logic flow. When an LLM chooses `const` versus `var`, it runs the same multi-billion-parameter forward pass that it uses to select between “luminous” and “radiant” in a poem — but the Zig decision carries one bit of information determined by a scoped dataflow fact, while the poetry decision carries 12-15 bits of genuine creative selection across thousands of plausible candidates.

When an LLM does arithmetic, it predicts digits one at a time through full forward passes. Each digit is a 3-4 bit selection through a pipeline designed for 15 bits. The error rate is 2-5% per operation and compounds through chains. A 500-position portfolio correlation matrix is impossible — the data alone exceeds any context window, and the arithmetic would be wrong even if it fit.

When an LLM maintains state across a conversation, it re-reads the entire history through attention on every turn. Turn 1 processes 6,000 tokens. Turn 10 processes 195,000 cumulative tokens. Turn 50 processes 3.9 million. The cost is quadratic in conversation length, and the model’s ability to find relevant prior information degrades as the history grows because attention dilutes across an ever-larger context.

When an LLM hedges — “approximately,” “it appears that,” “please consult a professional” — it is generating tokens with no computational basis, filling space where a confidence measurement should be. These tokens exist because the system has no mechanism for computing or reporting confidence as a value. They cost full forward passes and convey no information.

None of these are failures of the language model. They are consequences of an architecture that routes every output token, regardless of information content, through the same expensive prediction pipeline. The language model is being used as a very expensive, unreliable grammar engine, calculator, database, logic engine, and formatting system simultaneously, when each of those functions has a deterministic solution that is thousands to millions of times faster and exactly correct.

3. The Elimination Mechanism

Each category of wasted computation has a specific, mechanical replacement.

Structural tokens are eliminated by grammars. A grammar is a persistent template on a KB that declares fixed output structure with typed content slots. The grammar emits all structural tokens — braces, delimiters, headers, keywords, indentation, tags — directly, with zero forward passes and 100% correctness. The LLM fills content slots where judgment is required. Grammars nest recursively: a JSON grammar dispatches to entry

grammars, which dispatch to type-specific value grammars, which dispatch to element grammars. Each nesting level eliminates structural tokens and constrains the vocabulary at the next level. A categorical slot with 4 valid values reduces the softmax computation by $12,500 \times$ for that token position.

Grammar-directed generation is not constrained decoding, which still runs a forward pass per token and merely masks illegal candidates. It eliminates the forward pass entirely for structural tokens. The grammar produces them directly from the template. The distinction is between “predict from 50,000 candidates but mask 49,996 of them” and “emit the known token, skip prediction.”

Computation tokens are eliminated by builtins. The VDR system specifies 448 typed primitives across 25 categories: exact closed and active arithmetic, lift and rebase, comparison, rounding, number theory, list aggregates, Q-basis operations, functional remainders, discrete calculus, full linear algebra, probability and statistics, conversion, polynomial algebra, finite fields, Markov chains, graph algorithms, integer fast paths, bit operations, denominator management, text operations, collections, sets, mappings, time, identity, logic, graphs, and KB/constraint operations. Each invocation costs ~8 LLM tokens as a command token — a structured reference from a ~500-item vocabulary at ~6 bits per token. The computation executes in nanoseconds on exact integers. The LLM selects which computation to perform. The primitives perform it.

Command tokens achieve 99.2% error-free probability versus 86% for JSON function calling and ~60% for free-form code generation. The error rate difference comes from entropy: ~6 bits per command token versus ~15.6 bits per vocabulary token. Lower entropy means fewer bits to get wrong per token, and fewer tokens per invocation.

State reconstruction tokens are eliminated by Knowledge Bases. Data lives as facts at integer addresses in a scoped KB tree. Retrieving a fact is one integer-addressed query — $O(1)$ with the UUID, $O(\text{depth})$ with the dotted path. The LLM reads current state from KB, not from conversation history. Turn 100 costs the same as turn 1. There is no quadratic growth. There is no attention dilution. There is no context window overflow. Data that doesn’t fit in a context window — 1MB of JSON metrics, 10MB of documents, 500-position portfolios — sits in the KB and is accessed through builtins. The capability boundary moves from “fits in context window” to “fits in memory.”

Deduction tokens are eliminated by Prolog. A Prolog engine performs depth-first search with backtracking over exact integer facts. Unification uses exact comparison via cross-multiplication. The engine evaluates logical chains in microseconds, deterministically, producing the same result every time. A rule that took 200 tokens of prose reasoning — “if the latency shows a step function pattern and the throughput dropped by more than 25% and a deployment occurred within 5 minutes of the spike, then the probable cause is the deployment” — becomes a Prolog clause that fires in microseconds and produces a confidence of 85/100 as an exact VDR fraction.

Formatting tokens are eliminated by grammar templates. The same mechanism that handles JSON braces handles markdown tables, incident report structure, code documentation format, and any other structured output. The LLM provides content. The grammar provides presentation.

Hedging tokens are eliminated by computed confidence. Confidence propagates as exact VDR fractions through declared formulas. VDR computation: 1/1. Prolog derivation: 1/1. Database query: 98/100. API response: 85/100. Multiple agreeing sources: $1 - \prod(1 - C_i)$. LLM-generated content: 30/100 fixed floor. The fraction replaces the hedge. “The probable cause is deployment v2.3.1 (confidence: 92/100, from: deployment timing at 95/100 combined with latency pattern at 85/100)” carries more information in fewer tokens with exact provenance.

Prose tokens are partially eliminated by sentence templates. The LLM emits a semantic tuple — subject, verb, object, modifiers — at ~8 command tokens. Prolog matches against a library of ~5,000 sentence structure templates. The template fills slots with the semantic content and returns a complete sentence. English scaffolding — articles, prepositions, conjugation — is provided by the template at zero cost. When no template matches (the sentence structure is genuinely novel), the LLM full-generates as it does today. The template library grows through usage. The proportion of prose requiring full generation decreases over deployment lifetime.

4. Measured Elimination by Task

The token reduction has been measured through task decomposition against known LLM behavior, breaking each task into its component token categories and applying the mechanical elimination for each category.

SRE incident investigation: 25,100 tokens conventional, 769 tokens VDR — $33 \times$ reduction, 98.6% eliminated. The investigation is almost entirely metric retrieval, threshold comparison, timeline correlation, causal deduction, and formatted reporting. The LLM’s judgment contribution is deciding what to investigate and assessing the final finding.

Legal document review: 30,000 tokens conventional, 1,130 VDR — $26.5 \times$ reduction, 96.2%. Clause matching, regulatory comparison, obligation tracking, deadline extraction — all structural operations on addressed data.

Financial analysis: 15,000 tokens conventional, 600 VDR — $25 \times$ reduction, 96%. Arithmetic on exact VDR fractions, time series comparison, threshold checking, compliance verification.

Medical record analysis: 80,000 tokens conventional, 4,740 VDR — $16.9 \times$ reduction, 94.1%. Lab value comparison, medication interaction checking, guideline matching against KB facts.

Codebase migration: 100,000 tokens conventional (partial coverage — cannot hold 200 files simultaneously), 6,700 VDR (complete coverage) — $14.9 \times$ reduction, 93.3%. AST analysis, pattern matching, dependency tracking, test generation — all through builtins operating on addressed data.

Customer support: 500 tokens conventional, 150 VDR — $3.3 \times$ reduction, 70%. More prose, but substantial state management and knowledge retrieval from indexed KB.

Academic grading: 200,000 tokens conventional (partial — inconsistent rubric application across 150 essays), 57,230 VDR (complete — consistent rubric via Prolog rules) — $3.5 \times$ reduction, 71.4%.

Open conversation with minimal structure: 10-30% reduction from grammar-assisted formatting.

Poetry and creative writing: ~0% token reduction. Every token is creative judgment. Per-token hardware speedup still applies.

The floor for any task producing structured output is ~40%. The floor for any task involving data processing is ~70%. The floor for pure unstructured creative prose is the hardware speedup alone.

5. Hardware: Parity as the Baseline

The VDR arithmetic substrate does not require novel hardware. It runs on integer units that have been shipping in every CPU and GPU since 2017.

The Zig toy measures the instruction-level equivalence directly. VDR Q16 multiply-accumulate is a widening multiply ($i16 \times i16 \rightarrow i32$), accumulation in $i64$, and a right-shift epilogue extracting V and R. This is the identical instruction sequence used by INT8/INT16 quantized inference. The hardware does not know or care that the bits below the shift are being called “remainder” instead of being discarded. The throughput is the same. The energy is the same. The latency is the same.

On GPU, INT8 tensor cores on H100 operate at 1024 ops/SM/cycle — $2 \times$ the FP16 tensor core rate of 512 ops/SM/cycle. INT8 weights are half the size of FP16 weights, doubling effective memory bandwidth for single-batch autoregressive inference, which is memory-bandwidth-bound. The combination of $2 \times$ compute throughput and $2 \times$ memory bandwidth produces the projected $\sim 2 \times$ forward pass speedup for GEMM-dominated operations.

The larger advantage comes from eliminating the Special Function Unit bottleneck. The SFU on H100 handles $\exp$, $\log$, rsqrt , $\sin$, $\cos$, and division at 32 ops/SM/cycle — 1/16th of the FP16 tensor core rate and 1/32nd of the INT8 tensor core rate. Every conventional softmax evaluates $\exp()$ per element through the SFU. Every GeLU activation evaluates $\tanh$ through the SFU. Every layer norm evaluates rsqrt through the SFU. These are the non-GEMM operations that create pipeline bubbles while tensor cores wait.

VDR replaces all SFU-dependent operations with table lookups at full shared memory bandwidth or Barrett reduction at INT32 core rate. The quadratic softmax surrogate — each output equals the square of the shifted input divided by the sum of all squared shifted inputs — uses subtraction, squaring, and integer division. No transcendentals. No SFU. No warp divergence from data-dependent special values. The output sums to exactly 1 by construction.

Projected speedups on non-GEMM operations: softmax $3\text{-}4 \times$, GeLU/SiLU activation

$4\text{--}6 \times$, layer norm $2\text{--}3 \times$. Combined with GEMM at $\sim 2 \times$, the weighted forward pass speedup is approximately $2 \times$ on H100.

Energy per multiply-accumulate at 7nm (Horowitz ISSCC 2014, scaled): INT8 at 0.03 pJ, INT16 at 0.08 pJ, FP16 at 0.17 pJ, FP32 at 0.59 pJ. VDR Q16 is $2.6 \times$ less energy than FP16, $7.9 \times$ less than FP32 per MAC. At datacenter scale with hundreds or thousands of GPUs, the energy differential is a material operational cost.

Integer arithmetic provides bit-identical determinism as a structural property. Integer addition is associative. The order of accumulation does not affect the result. Two runs with the same seed on different hardware produce the same output at every bit. This is a mathematical property of the representation, not an engineering achievement. It cannot be removed. It eliminates the entire class of problems arising from float non-determinism: irreproducible training runs, non-deterministic test suites, compliance documentation for stochastic outputs, debugging that depends on reproducing exact conditions, and A/B testing noise from arithmetic variation.

All of this runs on hardware that has been in production since 2017. Volta V100 introduced INT8 tensor cores. Every NVIDIA GPU since then has INT8 and INT32 integer units. AVX-512 integer instructions ship on Intel Xeon since Skylake-X (2017). ARM NEON integer has been on every ARM core since ARMv8 (2011). Apple AMX INT8 has been on every Apple Silicon chip since M1 (2020). TPU INT8 systolic arrays have been available since TPU v3 (2018). No hardware development is required.

6. Scaling: Linear Versus Quadratic

The conventional LLM architecture processes conversation as a flat token sequence. Every generation step attends over the entire history. The cost of turn N is proportional to the sum of all tokens from turns 1 through N . For a conversation where each turn generates approximately the same number of tokens, total cost grows as $N \times (N+1)/2$ — quadratic in conversation length.

The VDR architecture stores state in Knowledge Bases at integer addresses. Each turn reads the current state it needs from KB queries, performs its work, stores results back to KB, and generates output. The cost of turn N is proportional to the work done on that turn, independent of how many turns preceded it. Total cost grows as $N \times C$ where C is the constant per-turn cost — linear in conversation length.

At turn 1, the ratio is modest: 23:1 (6,000 conventional versus 260 VDR for an SRE-class structured task). By turn 10, the ratio is 75:1. By turn 20, 133:1. By turn 50, 300:1. By turn 100, 588:1. The conventional system saturates — context fills with its own prior output, leaving zero capacity for new work. The VDR system never saturates because state occupies KB addresses, not context window positions.

This scaling advantage is independent of the token reduction. Even if VDR generated the same number of tokens per turn as conventional (which it does not — it generates 70-98%

fewer), the linear-versus-quadratic scaling would still produce a growing cost advantage over any multi-turn interaction.

The quality trajectories are also opposite. Conventional accuracy degrades as conversation length increases — attention dilutes across growing context, relevant information becomes harder to locate, and the model increasingly confuses current state with obsolete prior states. VDR accuracy improves because the KB accumulates verified findings. Each turn adds knowledge at stable integer addresses. Prior findings don't dilute or drift. They are addressable, exact, and provenanced.

7. Rule Accumulation: The System Improves Through Usage

Every session can create persistent Prolog rules, Python scripts, grammars, and compaction rules. These artifacts persist in the KB tree at the scope where they were created — session scope for ephemeral experiments, project scope for team-shared patterns, organizational scope for company-wide automation. Each artifact is inspectable (readable Prolog clauses with provenance), reversible (clean retraction), and composable (rules from different sources interact through structural unification automatically).

A Prolog rule costs 25-40 tokens to formalize. It replaces 150-300 tokens of conventional LLM reasoning per invocation. It breaks even on first reuse. At organizational scope with thousands of invocations, the amortized cost approaches zero.

The accumulation curve for SRE triage demonstrates the progression. Investigation 1: 329 total tokens, 15 rules available, 0% automated, 0% at Level 3. Investigation 10: 92 tokens, 64 rules available, 65% automated, 33% at Level 3. Investigation 50: 65 tokens, 140 rules available, 88% automated, 66% at Level 3. Investigation 100: 55 tokens, 185 rules available, 93% automated, 75% at Level 3.

Level 3 is pure Prolog execution with zero LLM involvement. The rules fire on data, evaluate conditions, produce findings, and store results — all through deterministic integer operations. The LLM is consulted only when the situation is genuinely novel — when no existing rule matches, when the evidence is ambiguous, when judgment about priority or communication is required.

This is the inversion of the conventional cost trajectory. Conventional systems pay the same cost every time for the same type of problem because they have no mechanism for encoding learned patterns into reusable deterministic procedures. VDR systems pay decreasing cost because each solved problem can become a rule that executes at zero LLM cost on all future encounters.

Negative accumulation prevents unbounded rule growth. Three automated hygiene rules detect and handle staleness: rules not fired in 90 days are flagged for review, rules with less than 20% success rate after 10+ firings are retracted immediately, and rules that have never successfully executed due to missing grants are flagged. Retraction is clean — provenance tracks what the rule did, what depends on it, and what changes when it is removed.

The bootstrap sequence for a new deployment follows defined stages. Seeded: four seed layers loaded (language templates, format grammars, operational rules, self-maintenance rules — approximately 23,400 entries, 1.5 MB, loads in under 620ms). Operating: first successful user interaction producing stored findings. Self-compacting: the system compacts known document types without external LLM involvement, reaching approximately 98% fact parity with external LLM processing after 200 documents. Self-extending: the system creates new grammars for novel document structures without guidance. Mature: compaction rules cover over 90% of incoming document types, operational rules cover over 80% of routine tasks.

8. Diffusion and Long-Chain Computation

Diffusion models create sequential arithmetic chains — each denoising step feeds its output as input to the next step. In floating-point arithmetic, each step contributes a rounding error of approximately one unit in the last place. Over N steps, the error accumulates linearly. For a 2-hour film at 24 frames per second with 50 denoising steps per frame, the chain is 8.64 million operations. Float64 accumulated drift at that chain length is approximately 1.9×10^{-8} per element — visible as color shift, flicker, and temporal inconsistency. Correction passes every few hundred steps add 5-8% computational overhead and introduce discontinuities.

VDR drift is structurally zero. Integer addition and multiplication are exact. The only approximation is Newton iteration for square roots of schedule coefficients, which produces an exact rational at each depth with an exact inspectable residual. At depth 10, the Newton residual is below 10^{-50} . This residual is fixed per square root evaluation — it does not compound through the chain. Error at cycle N equals error at cycle 1.

The DDIM deterministic roundtrip — forward diffusion followed by reverse denoising with a perfect noise predictor — has exactly zero error. The reverse process perfectly inverts the forward process because the arithmetic is exact. This is validated by 37 tests with 33 passing and 4 failing on normalization presentation issues with zero arithmetic errors.

Combined with the $\sim 2 \times$ throughput advantage from INT8 tensor cores and the elimination of correction passes, the projected advantage for video diffusion workloads is $2.1 \times$. For applications requiring platform-independent reproducibility — medical imaging, scientific visualization, forensic computation, regulatory compliance — the bit-identical determinism of VDR is not a performance advantage but a capability that float cannot provide at any cost.

9. Structural Safety at Zero Cost

Safety in the VDR architecture is not a feature added to the system. It is a consequence of how the system was built. Three components designed for other purposes — KB visibility (built for data scoping), grants (built for operation governance), and grammar

output validation (built for format correctness) — combine to produce safety properties that conventional systems achieve through behavioral training at ongoing cost.

KB visibility filters data before the LLM receives it. Each KB has a visibility level: public, internal, or owner-only. Every query checks the requesting user’s identity against each candidate KB’s visibility setting — an integer comparison inside the primitive call. A failed check means the KB is absent from the result set, not redacted. The LLM cannot be prompted to reveal data it never received.

Scope chain resolution limits which KBs are searchable. A query walks from the user’s position upward through ancestors to root. Sibling branches are structurally unreachable — the walk algorithm goes up and down, never sideways. An engineer in the engineering branch cannot reach HR data in the HR branch because the HR branch is a sibling, not an ancestor or descendant.

Grant authorization gates operational primitives. All 44 operational primitives (filesystem, compilation, execution, linting, network, process) require a positive credential grant. Default is denial. Grant state transitions are monotonic: active to expired, active to exhausted, active to revoked — never back. No re-increment. No un-revoke.

No input to the LLM modifies any integer involved in any access control check. Session `user_id` is set at authentication and stored at the session root KB (`-1.identity.user_id`) — it is immutable from the token stream. Prompt injection changes LLM intent but cannot change the `user_id` that the primitive layer reads. Role-play changes LLM self-concept but the primitive layer checks the session KB, not the LLM’s beliefs. Many-shot attacks shift behavioral baselines but the access checks are in the primitive layer, not the attention layer. Encoding attacks bypass pattern-based refusal but VDR safety is access control on data, not pattern matching on queries.

For data access, jailbreaking is not difficult or unlikely — it is impossible. The attack surface does not exist. What prompt injection can do: influence which builtins the LLM invokes and how it phrases prose output. What it cannot do: change the user ID, modify the scope chain, bypass visibility checks, execute operations without grants, or surface data the session is not authorized to see.

Session scoring provides contextual access control without LLM involvement. Input classification matches tokens against a classification KB using string matching primitives. Matches increment monotonic integer counters on the session KB — professional signals and harm signals separately. Prolog rules evaluate counter values against configurable thresholds. A professional chemist accumulates pharmacology, quantitative measurement, and clinical medicine signals over several turns, exceeding the professional threshold, and gains access to restricted chemistry KBs. A harm-intent user accumulates harm signals that cannot be erased by subsequent “good” turns because the counters are monotonic. Thresholds are tunable by one KB fact assertion with immediate effect and no retraining.

The entire safety mechanism costs zero LLM tokens. It is integer comparison, counter increment, and Prolog rule evaluation — operations that take nanoseconds and execute outside the LLM’s forward pass. Conventional behavioral safety costs 500-1500 tokens

per request in hedging, refusal reasoning, and safety disclaimers, and remains probabilistically bypassable.

10. The Session Model

Every session begins with -1 as its root KB. This is hardcoded, universal, and requires no lookup. All session-local state — counters, queues, LRU caches, locks, stacks, ring buffers, bitsets, scratch KBs, draft rules, identity, grants, scoring counters — lives under -1. Prolog rules reference `session_root` (resolved to -1 at rule load time as a compile-time constant) and work identically in every clone.

The sign bit of every ID encodes lifecycle semantics. Positive IDs are global: they persist beyond the session, they exist in the shared address space, other entities may attempt to access them subject to visibility and grants. Negative IDs are ephemeral: they exist only within a session lifecycle, they are purged when the session ends, no external entity can reference them. The term “ephemeral” rather than “local” is precise — both global and ephemeral data have locality (they live in the same structures), but ephemeral data has a bounded lifecycle that ends with the session.

In a dotted path like 17.4.5.37.-1.-4.-17, the positive segments traverse the shared KB tree with normal access control. The first negative segment (-1, session root) marks the transition to ephemeral space. Everything deeper is session-owned and session-scoped. No negative-to-positive transition is valid in a path — you cannot create persistent shared state inside an ephemeral workspace. Promotion from ephemeral to global is an explicit `KB_ASSERT` to a positive-ID path, subject to normal visibility and grant checks.

UUIDs follow the same convention. The best UUID algorithm for the deployment scale generates N bits. Shift right by one to clear the high bit. The high bit is 0 for global, 1 for ephemeral. For signed integers, this means ephemeral UUIDs are negative and global UUIDs are positive. One comparison instruction — checking the sign — distinguishes them. Collision resistance decreases by exactly one bit, which at i128 is immaterial.

Dual addressing serves different access patterns. Dotted paths support tree navigation: scope walks, child enumeration, subtree queries, inheritance resolution. UUIDs support direct access: $O(1)$ fact retrieval, counter operations, queue push/pop. Both carry the sign-bit lifecycle semantics. The lookup table maps between them. For reserved labels like `session_root`, the mapping is a compile-time constant with zero lookup cost.

A clone’s workspace is its -1 subtree. Snapshots capture -1 and everything under it. Clone spawn copies the -1 subtree, giving the new clone identical initial state with independent mutability. Drift management monitors counters under -1: turn count, context saturation, denominator drift, error rate. When thresholds are exceeded, the clone is killed and a fresh one spawns from the same snapshot. The knowledge persists in the positive-ID shared KBs. The working state dies with the clone. The system strengthens over time because valuable findings are promoted to positive-ID KBs while accumulated drift is discarded with the ephemeral session.

11. LLM Software and Server Software

The session model generalizes to a complete application development paradigm. A configured LLM session with loaded data, encoded Prolog rules, mounted KBs, and tuned parameters is a running application. Snapshotting it produces a deployable binary. Cloning it produces a running process. The seven data primitives — counter, queue, stack, ring buffer, bitset, LRU cache, lock — provide the coordination mechanisms that conventional distributed systems implement with semaphores, message queues, mutexes, and shared memory. Wiring KBs together through these primitives is application architecture.

Development happens through conversation. The user interacts with the LLM, loads data, tests scenarios, and encodes correct judgments as Prolog rules. When the application works, the user snapshots it. Development time for a simple FAQ chatbot is approximately 4 hours versus 2-4 weeks of conventional development. A full customer support chatbot takes approximately 10 hours versus 4-8 weeks. An SRE triage assistant takes approximately 12 hours versus 6-12 weeks. The speedup comes from configuring a session with data and rules rather than writing code, building infrastructure, deploying services, and implementing monitoring.

Network server software follows the same pattern with the addition of protocol grammars. Every protocol follows one architecture: grammar speaks wire format, Prolog rules process requests, KB tree stores state, grants enforce security, provenance provides audit. A port listener is a processor runner with a granted network primitive. Connection isolation uses the clone model — clone-per-request for stateless protocols, clone-per-session for stateful protocols. Protocol compliance is structural: the grammar cannot produce malformed output because structural tokens are template-determined, not predicted.

This pattern covers 44 cataloged protocols including HTTP, WebSocket, GraphQL, gRPC, SMTP, IMAP, DNS, DHCP, SSH, LDAP, MQTT, AMQP, Redis, Kafka, OAuth, SAML, and SIP. The KB tree naturally maps to protocol data models: DNS zones are KBs with record facts, email inboxes are user child KBs, MQTT topics are topic path KBs, LDAP distinguished names are KB tree paths, S3 buckets are bucket KBs with object children.

Security properties that require middleware in conventional servers are structural in VDR. Authentication is credential facts plus Prolog rules — no middleware to misconfigure. Rate limiting is counter comparison with exact VDR fraction thresholds — no float drift, no false threshold crossings. SQL injection does not exist because Prolog queries are typed and there is no SQL engine. XSS does not exist because grammars produce safe output by construction. Financial rounding does not exist: \$10,000.00 is [1000000, 100, 0] and 99.99% SLA is [9999, 10000, 0].

Development time for a static HTTP server is approximately 3 hours. A JSON REST API takes 7-13 hours. A full email stack (SMTP inbound, SMTP outbound, IMAP) takes 18-26 hours. An OAuth/OIDC provider takes 9-12 hours. Conventional equivalents range from 1-2 weeks to 3-6 months. The services improve through usage as request patterns

become Prolog rules, and they are updatable by fact assertion without redeployment.

12. The Compound Economics

The axes of improvement are independent. Each applies regardless of whether the others are present. When they compound across a real workload, the combined effect is multiplicative.

Axis 1: Per-token hardware speedup. $\sim 2 \times$ from INT8 tensor cores at $2 \times$ FP16 rate, half-size weights doubling memory bandwidth, and SFU bottleneck elimination. Applies to every workload including creative writing. This is the floor.

Axis 2: Token elimination. 70-98.6% for structured tasks. Immediate upon deployment. The bulk is structural tokens and computation tokens that should never have required forward passes. Applies to every workload that produces structured output, processes data, or maintains state.

Axis 3: Scaling behavior. Linear versus quadratic over multi-turn sessions. Ratio grows continuously: 23:1 at turn 1, 75:1 at turn 10, 300:1 at turn 50, 588:1 at turn 100. Applies to every multi-turn interaction.

Axis 4: Rule accumulation. 83% cost reduction over 100 investigations as patterns shift from L1 (full LLM) to L3 (pure Prolog). Applies to every workload where similar problems recur — which is most enterprise workloads.

Axis 5: Engineering cost elimination. Determinism removes non-deterministic testing infrastructure, compliance documentation for stochastic outputs, and debugging of irreproducible failures. Structural safety removes ongoing RLHF, red teaming, prompt injection defense, output filtering, and jailbreak patching. Exact arithmetic removes NaN/Inf debugging, gradient clipping tuning, loss scaling, epsilon parameter management, and drift correction passes. These are real engineering costs at every organization running LLMs in production.

For a single SRE investigation: axis 1 ($2 \times$) multiplied by axis 2 ($33 \times$) gives $66 \times$. The VDR-18 case study measures $71 \times$ including human time: \$0.39 versus \$27.58, 9 seconds versus 660 seconds wall clock.

For SRE over a 6-month deployment: axis 1 ($2 \times$) multiplied by axis 2 ($33 \times$) multiplied by axis 3 ($20 \times$ average over typical session lengths) multiplied by axis 4 ($6 \times$ from rule accumulation) gives approximately $8,000 \times$ on LLM compute for mature structured workloads.

For customer support: axis 1 ($2 \times$) multiplied by axis 2 ($3.3 \times$) multiplied by axis 3 ($5 \times$ over typical session lengths) multiplied by axis 4 ($2 \times$ from rule accumulation) gives approximately $66 \times$ for moderate-structure workloads.

For creative writing: axis 1 ($2 \times$) only. Grammar saves some structural tokens in formatted output but the core work is irreducible LLM judgment.

At datacenter scale with a blended workload mix, a conservative $30 \times$ average reduction across task types: 420 GPUs serving a given workload drop to 14. Annual GPU cost at \$30K per GPU per year drops from \$12.6M to \$420K. Annual energy at 700W per GPU drops from 2,570 MWh to 86 MWh. Annual energy cost at \$0.10/kWh drops from \$257K to \$8.6K.

These are pre-optimization baselines. Every projection assumes first-generation kernel implementations at 75-85% utilization. The basis selection has not been tuned per workload, per layer, or per head. The R depth has not been profiled in production to determine whether it flattens to a fixed-width representation (eliminating tree traversal overhead). Grammar matching has not been compiled to skip Prolog evaluation for common patterns. Type dispatch has not been specialized for dominant operand type combinations. The optimization space is unexplored because there is no production implementation to profile. Every optimization that profiling reveals is pure upside on top of the conservative numbers presented here.

13. What Is Built, What Is Projected, What Is Unbuilt

The VDR system exists at three levels of validation.

Built and validated. The Python library `vdr-math` 0.1.0 shipped on PyPI with 921 tests across 38 domains and zero arithmetic errors. The Zig toy implementation measures instruction-level equivalence with quantized inference. The toy LLM runs a complete transformer pipeline — forward pass, backpropagation, weight updates, attention, softmax, sampling — in exact integer arithmetic with 9 verification tests passing including bit-identical determinism and exact weight update algebraic correctness. The diffusion module verifies zero-drift DDIM roundtrips and non-growing multi-cycle error. The example programs run and produce the outputs claimed: exact Hilbert matrix inverse, Newton square root with inspectable residuals, Q335 transcendental arithmetic with identity verification.

Projected from known operations. GPU forward pass timing is derived from published H100 tensor core throughput (989 TFLOPS FP16, ~989 TOPS INT8 equivalent) and published SFU rates (32 ops/SM/cycle). Energy per operation is from published silicon measurements (Horowitz ISSCC 2014). Token reduction ratios are from task decomposition against measured LLM behavior, breaking tasks into their component token categories. Prolog query performance is from textbook algorithm analysis on known hardware. Rule accumulation curves are from the operational deployment specification with measured seed layer sizes. All projections use known operation costs on shipping hardware.

Specified but unbuilt. GPU kernels (8 kernel types across a 12-25 month development roadmap in 4 phases from functional correctness through near-peak optimization). KB infrastructure (26-field struct, scoped tree, integer addressing, dual path/UUID addressing). Prolog engine (depth-first search with backtracking, frontier-based batched joins for GPU, scope-filtered unification). Grammar system (persistent KB field, recursive nesting, auto-generated extraction/display/usage grammars, scoring and matching). Session

management (snapshot, clone, drift thresholds, -1 session root, ephemeral/global sign-bit lifecycle). The complete system is specified at 65 modules, approximately 20,500 lines, across 5 incremental build stages with cumulative test targets from 150 to 1,250 tests.

Every unbuilt component is composed of individually well-understood operations. Integer comparison, hash table lookup, depth-first tree search, template string substitution, bounded queue push/pop, atomic counter increment — these are not novel algorithms. They are standard computer science implemented on hardware that has been available for years. The integration risk is real — how the components interact under production load, how kernel utilization progresses through the optimization phases, whether the grammar system covers enough output structure to achieve projected token reductions. But the individual operations will perform as documented because they are the same operations that existing systems already execute billions of times per second.

The appropriate framing is not “does this approach work” — the arithmetic is validated, the instruction equivalence is measured, the token elimination is mechanical. The appropriate framing is “how long does it take to build and optimize the production system, and how close do the realized numbers come to the conservative projections.” The answer to the first question is the engineering roadmap. The answer to the second question can only come from building and measuring.

14. Industry Context

The AI industry is spending over one trillion dollars on infrastructure in 2026. Power consumption is becoming a physical, regulatory, and economic constraint on continued scaling. The dominant response is building more datacenters, designing more efficient chips, and optimizing float operations at diminishing marginal returns.

The float substrate has fundamental properties that engineering cannot change. Float addition is not associative — this is a mathematical property of the representation, not an implementation limitation. Softmax over float will never sum to exactly 1. Drift will always accumulate over long chains. NaN and Inf will always be possible. Denormals will always cause performance variation or silent precision loss. No amount of scaling, optimization, or hardware improvement changes these properties because they are consequences of the representation itself.

The trajectory of industry workloads amplifies every VDR advantage. Longer outputs (video generation, extended reasoning chains) increase drift exposure. More structured tasks (enterprise automation, agentic workflows) increase the fraction of tokens that are mechanical rather than creative. Higher reliability requirements (regulatory compliance, financial applications) decrease tolerance for non-determinism. Tighter cost constraints increase the value of 20-70 $\times$ compute reduction.

VDR addresses these through a different substrate rather than optimization of the existing one. The cost of investigation is bounded: a small engineering team working for months on existing hardware. If the economics don’t materialize at production scale, the cost is engineer-months. If they do materialize and a competitor deploys first, the cost is

operating at $20\text{-}70 \times$ less value per GPU-dollar with no path to close the gap through float optimization, because the gap is architectural.

References

VDR-1 through VDR-32: Complete paper series establishing exact arithmetic (VDR-1, 2, 3), exact LM pipeline (VDR-4), knowledge bases and Prolog (VDR-5), primitives and environments (VDR-6), lifecycle management (VDR-7), data primitives and sessions (VDR-8), orchestrated inference (VDR-9), IOSE model and builtins (VDR-10), implementation blueprint (VDR-11), grammar-directed compaction (VDR-12), physical computation (VDR-13), consolidated specification (VDR-14), prompt optimization (VDR-15), structural safety (VDR-16), alignment (VDR-17), GPU performance (VDR-18), self-extension (VDR-19), operational deployment (VDR-20), FPGA accelerator (VDR-21), dedicated silicon (VDR-22), functional remainder hardware (VDR-23), LLM software (VDR-24), LLM server software (VDR-25), diffusion (VDR-26), extended domains (VDR-27, 28), SIMD and GPU performance (VDR-29), economics (VDR-30), toy LLM in Python (VDR-31), toy LLM in Zig (VDR-32).

[vdr-math](#) 0.1.4: Python reference implementation. PyPI package. MIT license. 921 tests, zero arithmetic errors.

HOWL-VDR-33-2026 — Appendices

Appendix A: Instruction Equivalence Detail

The claim that VDR Q16 compiles to the same instructions as quantized inference rests on a step-by-step comparison of the multiply-accumulate sequence.

Step	VDR Q16	INT8 Quantized	Difference
Weight load	load i16	load i8	VDR $2 \times$ memory per weight
Activation load	load i16	load i8 or i16	Same or VDR $2 \times$ memory
Multiply	$i16 \times i16 \rightarrow i32$	$i8 \times i8 \rightarrow i16$ or $i32$	Same instruction class
Accumulate	$i64 += i32$	$i32 += i16$	Same instruction class
Epilogue	right shift 16	float multiply by scale	VDR simpler (shift vs multiply)
Store result	store i16	store float16	Same bandwidth
Remainder	store i16 (optional)	discarded	VDR tracks what quantization loses

The compute cost is identical. The memory cost is $2 \times$ for Q16 versus INT8, equal for Q16 versus INT16. The epilogue is cheaper for VDR because a right shift replaces a float dequantization multiply. In the Zig toy, remainders are zeroed (discarded) during inference, matching quantized behavior exactly. The option to retain them exists but is not exercised during inference — it is available for training and validation where tracking the residual matters.

Appendix B: Forward Pass Operation Count (Toy Model)

Measured from the Zig toy at 181 parameters, $D = 2^{16}$, single-block single-head causal transformer with 5-token vocabulary, embedding dim 4, FFN hidden 8, context length 4.

Stage	Multiplies	Adds	Shifts	Compares	Total Ops
Embedding (token + pos)	0	16	0	0	16
Wq projection	64	48	16	0	128
Wk projection	64	48	16	0	128
Wv projection	64	48	16	0	128
Attention scores (10 causal)	40	30	10	6	86
Softmax surrogate	20	24	0	16	60
Softmax division	0	4	0	0	20
Attention mix	64	48	16	0	128
Wo projection	64	48	16	0	128
Residual add (attention)	0	16	0	0	16
FFN layer 1 (4→8)	128	96	32	0	256
ReLU	0	0	0	32	32
FFN layer 2 (8→4)	128	96	16	0	240
Residual add (FFN)	0	16	0	0	16
Output projection (4→5)	80	60	20	0	160
Total forward	716	598	158	54	1,542

At 688 ns measured: 0.45 ns per operation, 1.57 cycles per operation at 3.5 GHz. Multiplies dominate at 46.4%. FFN layers account for 32.2% of total operations. The backward pass adds 3,013 operations (1,361 multiplies, 877 adds, 689 shifts, 86 compares). Combined forward + backward + SGD: 4,555 operations per training step at 1,159 ns measured, giving 0.254 ns per operation — higher throughput on backward because outer products have no loop-carried dependencies.

Appendix C: Type Width Map

The Zig toy uses mixed integer widths, matching the pattern specified for production in VDR-29.

Data	Stored Width	Compute Width	Notes
Weights	i16	i64 (during matmul accumulation)	Narrowed via »16 to i16 for output
Activations	i16	i64 (during dot product)	Same narrowing pattern
Probabilities	i32	i32	Never narrowed to i16; single probability can equal D=65536
Shifted scores	i32	i32	Never narrowed; causal mask fill can exceed i16 range
Gradients	i16 (stored)	i32 (accumulated) → i64 (applied)	Narrowed at storage boundary
Accumulators	register-only i64	i64	Never stored in arrays
Loss	i64 scalar	i64	Never stored in arrays

The pattern is consistent: store at the narrowest width that preserves the value range, compute at the width needed to prevent overflow, narrow at storage boundaries via right shift. This is identical to how quantized inference systems handle intermediate precision — the difference is that VDR names the bits below the shift “remainder” instead of discarding them.

Appendix D: Overflow Events and Resolutions

Discovered during Zig toy development. Each overflow traces to an intermediate value that is semantically Q16 but numerically exceeds the i16 range.

Source	Cause	Resolution	Production Implication
Embedding addition	Two i16 values sum exceeds ± 32767	Reduced init scale to ± 4096	Use Q32 embedding or init-scale discipline
Softmax shifted scores	Score minus causal mask fill exceeds i16	Widened shifted scores to i32	Shifted scores always i32 intermediate
Linear output	Dot product / D exceeds i16 after 50K steps	Clamped to ± 32767	Use Q32 storage or weight decay
Probability values	Single probability can equal D=65536 > i16 max	Stored probabilities as i32	Probabilities always i32 intermediate

The consistent resolution is: use the next-wider integer type for intermediate values, narrow at the storage boundary. This is the same engineering pattern used in every quantized inference implementation.

Appendix E: Saturation Events Over Training Duration

At Q16 (4.8 decimal digits precision), sustained training eventually saturates the i16 value range. This is the Q16 equivalent of gradient explosion in float training.

Step Range	Saturation Events per Step	Effect on Training
0–1,000	0	Loss decreasing normally
1,000–10,000	0–1	Slight loss flattening
10,000–25,000	1–5	Loss plateau
25,000–50,000	5–20	Loss oscillating (gradient information lost at clamp)

Production mitigations: weight decay, gradient clipping (exact integer comparison), learning rate warmdown, or Q32 storage for parameters that accumulate over many steps. The first 40 steps used for functional verification are unaffected. For production training, the VDR-29 specification assigns Q64 (i64) to gradient accumulation, providing ~19.3 decimal digits of headroom.

Appendix F: Memory Layout at Scale

Model Size	Weight Bytes (i16)	Fits In	Per-Param Cache Penalty vs L1
181 params (toy)	572 B	L1 (32 KB)	1 × (baseline)
10K	20 KB	L1	1 ×
100K	200 KB	L2 (256 KB)	~2 ×
1M	2 MB	L3 (8 MB)	~3 ×
100M	200 MB	HBM	~30 ×
7B	14 GB	HBM (1 × A100)	~30 ×
70B	140 GB	HBM (2 × A100)	~30 ×

Cache penalties are identical for float and VDR at the same element width. VDR Q8 weights (1 byte per param) match INT8 quantized at every cache level. VDR Q16 weights (2 bytes per param) match FP16 at every cache level. The per-parameter cost of 3.80 ns measured at L1 scales predictably through the memory hierarchy.

Appendix G: Energy Per Operation at 7nm

Source: Horowitz, ISSCC 2014, scaled to 7nm process node.

Operation	Energy (pJ)
INT8 multiply	0.03
INT16 multiply	0.08
INT32 multiply	0.24
FP16 multiply	0.17
FP32 multiply	0.59
INT16 add	0.008
FP16 add	0.06
FP32 add	0.14

Energy per 7B forward pass by arithmetic type:

Arithmetic	Energy per MAC (pJ)	Total Forward (mJ)	Ratio vs INT8
FP32	4.6	64.4	$7.4 \times$
FP16	1.5	21.0	$2.4 \times$
INT16 (VDR Q16)	0.58	8.1	$0.93 \times$
INT8 (quantized)	0.23	3.2	$0.37 \times$
INT8 weights $\times$	0.41	5.7	$0.66 \times$
INT16 acts (VDR Q8/Q16)			

VDR Q16 uses $2.6 \times$ less energy than FP16 and $7.9 \times$ less than FP32 per multiply-accumulate. These figures exclude memory access energy, which is identical for same-width formats.

Appendix H: Determinism Comparison Across Frameworks

Framework	Same Seed Same HW	Same Seed Diff HW	Distributed Diff Order
PyTorch float32 (default)	No (cuDNN non-det)	No	No
PyTorch float32 + det mode	Mostly	No	No
PyTorch float16	No	No	No
JAX float32	Yes (single device)	No	No
llama.cpp INT8	No (float dequant step)	No	N/A
VDR Python Q32	Yes	Yes	Yes
VDR Zig Q16	Yes	Yes	Yes

VDR is the only entry achieving determinism across all three columns. The property is structural: integer addition is associative, so accumulation order does not affect the result. This cannot be achieved by configuration because it is a mathematical property of the representation. It cannot be removed for the same reason.

Appendix I: Quantized System Comparison

System	Weight Fmt	Activation Fmt	Matmul Unit	Softmax	Deterministic	Sum-to-1 Exact
GPTQ	INT4	FP16	FP16 tensor core	FP16 exp	No	No
AWQ	INT4	FP16	FP16 tensor core	FP16 exp	No	No
llama.cpp Q4 0	INT4 + FP16 scale	FP32	FP32 scalar	FP32 exp	No	No
llama.cpp Q8 0	INT8 + FP16 scale	FP32	FP32 scalar	FP32 exp	No	No
SmoothQuant	INT8	INT8	INT8 tensor core	FP16 exp	No	No
ONNX INT8	INT8 + zero point	INT8	INT8 GEMM	FP32 exp	No	No
VDR Q16	INT16	INT16	INT16 widening	Quadratic (integer)	Yes	Yes
VDR Q8/Q16	INT8	INT16	INT8 × INT16 widening	Quadratic (integer)	Yes	Yes

Every existing quantization system converts to float for softmax. VDR stays integer end-to-end. The VDR Q8/Q16 weight format is byte-identical to SmoothQuant weight storage; the difference is the arithmetic treatment of the remainder (tracked versus discarded) and the softmax implementation (integer quadratic surrogate versus float exponential).

Appendix J: Softmax Cost Comparison

Method	Total Ops (vocab=5)	Ratio
Quadratic surrogate (integer)	19	1 ×
Taylor exp depth 8	94	4.9 ×
Taylor exp depth 16	174	9.2 ×

Method	Total Ops (vocab=5)	Ratio
--------	---------------------	-------

The quadratic surrogate also eliminates the exp lookup table (~8 KB at Q16 bounded range). On GPU, the surrogate runs at full INT32 core rate (64 ops/SM/cycle on H100) without touching the SFU (32 ops/SM/cycle). For larger vocabularies, the grammar-constrained decode reduces the effective vocabulary from 50,000+ to 2-200 candidates for structural tokens, making the softmax cost per structural token negligible regardless of method.

Appendix K: H100 Hardware Unit Throughput

Unit	Throughput (ops/SM/cycle)	Role in Float Pipeline	Role in VDR Pipeline
FP16 Tensor Cores	512	Matrix multiply-accumulate	Not used
INT8 Tensor Cores	1024	Not used (or dequant path)	Matrix multiply-accumulate ($2 \times$ FP16 rate)
FP32 CUDA cores	128	General float ops	Not used
INT32 CUDA cores	64	General integer ops	Softmax, activation, layer norm, KB ops, Prolog
SFU	32	exp, log, rsqrt, sin, cos, division	Not used (eliminated by table lookup + Barrett)

The SFU at 32 ops/SM/cycle is 1/16th of FP16 tensor core rate and 1/32nd of INT8 tensor core rate. Every operation that hits the SFU in the float pipeline — softmax exp, GeLU tanh, layer norm rsqrt — is a pipeline bottleneck where tensor cores sit idle. VDR eliminates the SFU from the critical path entirely.

Appendix L: Projected Forward Pass Timing (7B, H100)

Component	FP16 (ms/layer)	VDR INT8 (ms/layer)	Speedup
QKV projection	0.83	0.42	$2.0 \times$
Attention GEMM	0.55	0.28	$2.0 \times$
Softmax	0.036	0.009	$4.0 \times$
Attention output projection	0.28	0.14	$2.0 \times$
FFN up + gate	1.10	0.55	$2.0 \times$
GeLU activation	0.008	0.002	$4.0 \times$

Component	FP16 (ms/layer)	VDR INT8 (ms/layer)	Speedup
FFN down projection	0.55	0.28	2.0 ×
Layer norm (× 2)	0.008	0.004	2.0 ×
Residual add (× 2)	0.002	0.003	0.7 ×
Per-layer total	3.37	1.69	2.0 ×
Full forward (32 layers)	~110	~55	~2.0 ×

Residual add is the one operation where float is faster (single add versus V add + R add + carry check). It accounts for less than 0.1% of pipeline compute. GEMM dominates at ~85% of layer time, where the 2 × advantage from INT8 tensor cores at double the FP16 rate determines the overall speedup.

Appendix M: GPU Utilization Comparison

Operation	Conventional Float	VDR Integer	Reason for Difference
Matrix multiply	85-95% (tensor core)	60-80% (integer ALU, first-gen kernels)	cuBLAS decades of optimization; VDR kernel maturity gap
Softmax	40-60% (SFU transcendental divergence)	80-95% (integer surrogate, uniform)	No transcendentals, no warp divergence
KB scan	N/A	90%+ (columnar, coalesced)	Standard GPU database operation pattern
Scope filter	N/A	95%+ (bitset test)	One bit operation per element
Prolog unification	N/A	70-85% (minor divergence on term types)	Frontier-based batched joins, mostly uniform
Grammar-constrained decode	N/A (always full vocab)	95%+ (tiny candidate set)	Orders of magnitude less work per token
Provenance append	N/A	90%+ (atomic bump, bulk copy)	Minimal synchronization
Counter/bitset ops	N/A	95%+ (atomic int)	Single instruction per element

The matrix multiply utilization gap (85-95% versus 60-80%) is an engineering gap, not an architectural limitation. The four-phase kernel development roadmap targets 75-85% at Phase 3 (competitive performance, 3-6 months after functional correctness) and 85-95% at Phase 4 (near-peak, 6-12 months). The headroom from Phase 3 to Phase 4 represents pure engineering upside not included in the projections.

Appendix N: Drift Accumulation Over Chain Length

Chain Length	Float64 Drift per Element	VDR Drift	Context
1	$\sim 1 \times 10^{-15}$	0	Single operation
50	$\sim 5 \times 10^{-14}$	0	Single image generation
1,000	$\sim 1 \times 10^{-12}$	0	Short video clip
10,000	$\sim 1 \times 10^{-11}$	0	7 minutes at 24fps
100,000	$\sim 1 \times 10^{-10}$	0	1.2 hours
1,000,000	$\sim 1 \times 10^{-9}$	0	4.6 days continuous
8,640,000	$\sim 1.9 \times 10^{-8}$	0	2-hour film (24fps $\times$ 50 steps)
25,920,000	$\sim 2.6 \times 10^{-7}$	0	2-hour film, 3 render cycles

VDR drift is structurally zero — a mathematical property of integer arithmetic, not an empirical measurement approaching zero. The Newton sqrt residual (below 10^{-50} at depth 10) is constant per evaluation and does not compound through the chain. Float drift grows linearly with chain length and requires periodic correction passes (5-8% computational overhead) that introduce discontinuities.

Appendix O: Precision Comparison Across Approaches

Approach	Precision	Drift per Step	Drift at 1000 Steps	Reproducible	Inspectable
Float16	$\sim 10^{-3}$	$\sim 10^{-3}$	$\sim 10^0$ (unusable)	No	No
Float32	$\sim 10^{-7}$	$\sim 10^{-7}$	$\sim 10^{-4}$	No	No
Float64	$\sim 10^{-15}$	$\sim 10^{-15}$	$\sim 10^{-12}$	No	No
Float128	$\sim 10^{-33}$	$\sim 10^{-33}$	$\sim 10^{-30}$	Platform-dependent	No
Kahan summation	$\sim 10^{-15}$	$\sim 10^{-30}$	$\sim 10^{-27}$	No	No
VDR depth 10	$\sim 10^{-50}$ (Newton only)	0 (rational ops)	$< 10^{-50}$ (constant)	Yes	Yes

Approach	Precision	Drift per Step	Drift at 1000 Steps	Reproducible	Inspectable
VDR depth 20	$\sim 10^{-100}$ (Newton only)	0 (rational ops)	$< 10^{-100}$ (constant)	Yes	Yes

The “Inspectable” column refers to whether the error at any step is an exact, queryable value rather than a statistical bound. In VDR, the Newton residual x^2-2 at any depth is an exact rational that can be printed, compared, and reasoned about. In float, the accumulated error is unknown — it depends on the operation sequence, platform, and compiler flags in ways that cannot be determined after the fact.

Appendix P: Token Reduction Breakdown by Category

Detailed decomposition for three representative tasks.

SRE Investigation (5 turns)

Category	Conventional Tokens	VDR Tokens	Elimination Mechanism
State reconstruction	2,250	0	KB query by integer address
Computation	3,000	0	Exact integer builtins
Deduction	2,250	0	Prolog evaluation
Formatting	3,000	0	Grammar templates
Hedging	1,500	0	Computed confidence fractions
Command tokens	0	80	~10 builtin invocations at ~8 tokens
Judgment	1,500	70	LLM decides what to investigate
Prose	1,500	60	Semantic tuples + sentence templates
Total	15,000	210	98.6% reduction

Financial Portfolio Analysis

Category	Conventional Tokens	VDR Tokens	Elimination Mechanism
Data parsing	5,000	0	Builtins parse at integer addresses
Computation	5,000	0	Exact VDR arithmetic, zero drift
Formatting	3,000	0	Grammar templates
Hedging	1,000	0	Computed confidence fractions
Command tokens	0	200	~25 builtin invocations
Judgment	500	100	LLM selects analysis approach
Prose	500	300	Assessment and recommendations
Total	15,000	600	96% reduction

Academic Grading (150 essays)

Category	Conventional Tokens	VDR Tokens	Elimination Mechanism
Rubric reconstruction	30,000	0	Rubric as Prolog rules in KB
Computation	0	0	N/A
Consistency overhead	65,000	30	Prolog rules apply consistently; one script
Formatting	40,000	0	Grammar templates per essay
Command tokens	0	200	Rubric application calls
Judgment	45,000	45,000	LLM reads each essay (irreducible)
Prose	20,000	12,000	Feedback via sentence templates
Total	200,000	57,230	71.4% reduction

The grading task demonstrates the floor for prose-heavy work. The LLM must read and assess each essay — that judgment is irreducible. But the rubric never drifts (Prolog rules versus re-reading rubric text on each essay), formatting is free (grammar templates), and feedback prose partially templates (common phrases like “strong thesis statement” or “needs more supporting evidence” recur across essays).

Appendix Q: Scaling Over Conversation Length

Turn	Conventional Cumulative Tokens	VDR Cumulative Tokens	Ratio	Conventional Context Available
1	6,000	260	23:1	~40%
5	60,000	1,300	46:1	~10%
10	195,000	2,600	75:1	~4.5%
20	690,000	5,200	133:1	~2.4%
50	3,900,000	13,000	300:1	Saturated (0%)
100	15,300,000	26,000	588:1	Saturated

Conventional cost is quadratic: each turn re-reads all prior turns through attention. The cumulative token count is $T \times (1+2+\dots+N) = T \times N \times (N+1)/2$ where T is the average tokens per turn.

VDR cost is linear: each turn reads current state from KB at constant cost. The cumulative token count is $N \times C$ where C is the per-turn cost.

The “Context Available” column shows the fraction of the context window available for new work on the current turn, assuming a 128K token context window. By turn 50, the conventional system has consumed its entire context with prior history, leaving zero capacity for new work. The VDR system’s context consumption is constant per turn because prior state is in KB, not in the context window.

Appendix R: Rule Accumulation Over Deployment

SRE Triage Accumulation

Investigation	L1 Tokens	L2 Tokens	L3 Tokens	Total	L3%	Rules Avail- able	Auto- Triage %
1	280	49	0	329	0%	15	0%
2	78	41	8	127	6%	19	25%
5	55	40	15	110	14%	34	45%
10	30	32	30	92	33%	64	65%
20	18	22	38	78	49%	95	78%
50	10	12	43	65	66%	140	88%
100	6	8	41	55	75%	185	93%
200	4	5	39	48	81%	220	—
500	3	4	36	43	84%	260	—

L1 drops from 280 tokens to 3 — a $93 \times$ reduction in full LLM judgment. L3 rises from 0 to 36 tokens of pure Prolog execution at zero LLM cost. The total drops from 329 to 43 — an 87% reduction from accumulation alone, independent of the structural token elimination which already removed 98.6% of conventional tokens.

Cross-Application Accumulation Comparison

Application	Investigation 1	Investigation 10	Investigation 50	Investigation 100	Auto- Triage at 100
SRE triage	329	92	65	55	93%
Customer support	200	80	50	40	85%
Document processing	150	60	35	25	92%
Compliance review	250	100	55	45	88%

All applications show the same pattern: rapid cost reduction in the first 10-20 encounters as common patterns become rules, followed by diminishing but continuing improvement as less common patterns are encountered and formalized. The plateau reflects the fraction

of genuinely novel situations that require LLM judgment — approximately 5-15% of encounters depending on domain diversity.

Appendix S: Capability Boundary Comparison

Data volumes that conventional LLMs cannot process versus VDR handling.

Data Type	Size	Token Equivalent	Conventional LLM	VDR	VDR Mechanism
JSON metrics	1 MB	~300K tokens	Cannot load	Routine	Builtin fetch + parse to KB
Document (docx)	10 MB	~2.5M tokens	Cannot load	Routine	Builtin read + script parse
CSV dataset	5 MB	~1.2M tokens	Cannot load	Routine	Builtin read + CSV parse
Code repository	200 files	~500K tokens	Cannot hold simultaneously	Routine	Tree walk + per-file read
Knowledge base	2,000 articles	~5M tokens	Cannot search	Routine	Glob + indexed KB query
Portfolio	500 positions	~25K tokens	Overflows useful context	Routine	Parse + VDR arithmetic
Correlation matrix	500×500	Cannot compute	Exact	vec dot + mat new builtins	
Time series	90 days $\times$ 50 series	Cannot process all	Exact	discrete derivative builtin	
Cross-reference	2,000 documents	Cannot hold	Constant-time	kb query across	

The boundary is architectural: conventional LLMs can only process data that fits in the context window as serialized tokens. VDR processes data at integer addresses through builtins. The context window holds the LLM’s current reasoning, not the data being reasoned about.

Appendix T: Structural Token Percentages by Output Format

Format	Structural %	Grammar-Providable	Error Classes Eliminated
JSON object (20 fields)	55%	110 of 200 tokens	Mismatched braces, missing commas, unclosed strings
Markdown table (20 rows)	60%	300 of 500 tokens	Misaligned columns, missing pipes
CSV (50 rows)	50%	200 of 400 tokens	Unescaped commas, inconsistent quoting
Python function (30 lines)	40%	120 of 300 tokens	Missing semicolons, mismatched brackets
Zig module	70-80%	~750 of 1000 tokens	Syntax errors of all kinds
DOCX/XML	80-90%	~850 of 1000 tokens	Tag mismatch, namespace errors
Compacted table	80%	~800 of 1000 tokens	Pipe alignment, header mismatch
Structured prose with data	25-30%	~500 of 2000 tokens	Formatting inconsistencies
Incident report	50%	400 of 800 tokens	Template deviations
English prose (no structure)	~5%	Punctuation only	Minimal

Every grammar breaks even on first use. A JSON grammar costs ~15 tokens to define and saves ~110 tokens per use. A markdown table grammar costs ~12 tokens and saves ~30 tokens per row. Grammars persist in the KB tree, inherit through scope, and are reused across sessions at zero marginal cost.

Appendix U: Command Token Entropy Comparison

Mode	Vocabulary Size	Bits per Token	Tokens per Action	Error-Free Probability
Command token	~300 names + ~200 paths	~6	~8	~99.2%
JSON function call	50,000+	~15.6	~30	~86.0%
Free-form code	50,000+	~15.6	~50	~60.5%

Mode	Vocabulary Size	Bits per Token	Tokens per Action	Error-Free Probability
Natural language reasoning	50,000+	~15.6	~100	~13.3%

Command tokens achieve $10 \times$ lower entropy per token than JSON function calling and $16 \times$ lower than natural language. The error-free probability compounds per token: at 99.2% per token over 8 tokens, the probability of a fully correct invocation is 93.8%. At 86% per token over 30 tokens for JSON, the probability of fully correct JSON is approximately 1%. This is why function calling fails at non-trivial complexity — the per-token error rate compounds over the longer token sequences that structured formats require.

Appendix V: Context Window Utilization

Turn	Conventional Total Context	Conventional Available %	VDR Total Context	VDR Available %	Efficiency Ratio
1	5,000	40%	800	50%	$1.25 \times$
5	18,000	10%	1,000	45%	$4.5 \times$
10	33,000	4.5%	1,200	40%	$8.9 \times$
20	63,000	2.4%	1,500	33%	$13.8 \times$
50	153,000	Saturated (0%)	1,500	33%	∞

“Efficiency ratio” measures the fraction of context window containing useful current-turn information versus accumulated history. Conventional systems become progressively less efficient as their own prior output consumes context capacity. VDR systems maintain stable efficiency because prior state lives in KB, not in the context window.

Appendix W: Workday Economics by Domain

Aggregated across typical daily task mix per domain.

Domain	Conventional Daily Tokens	VDR Daily Tokens	Ratio	Highest-Ratio Task	Lowest-Ratio Task
SRE	640,000	26,000	24.6:1	Postmortem (120:1)	Ad-hoc query (5:1)
Legal	152,000	11,690	13.0:1	Contract review (26.5:1)	Memo drafting (3:1)

Domain	Conventional Daily Tokens	VDR Daily Tokens	Ratio	Highest- Ratio Task	Lowest- Ratio Task
Research	156,000	10,740	14.5:1	Synthesis (16.9:1)	Report drafting (3.3:1)
Development	130,000	19,675	6.6:1	Migration (14.9:1)	Feature implemen- tation (2.5:1)
Finance	200,000	15,000	13.3:1	Portfolio analysis (25:1)	Commentary (3:1)
Education	250,000	60,000	4.2:1	Statistics (∞)	Essay grading (3.5:1)
Support	50,000	8,000	6.3:1	KB setup (2500:1 amortized)	KB query (3.3:1)

The pattern is consistent: data-heavy and computation-heavy tasks achieve 10:1 to 100:1+ ratios. Prose generation tasks achieve 2:1 to 4:1 ratios. The floor never drops below 2:1 because grammar always saves some structural tokens in any formatted output.

Appendix X: SRE Case Study — Detailed Phase Breakdown

From the VDR-18 GPU performance analysis.

Phase	Description	Conv. Tokens	VDR Tokens	VDR GPU Ops	VDR Wall (ms)	Conv. Data Cover- age	VDR Data Cover- age
Data acqui- sition	Prometheus fetch + parse	12,200	72	1.2M	750	25% (50/200 end- points)	100%
Filtering + thresh- old	Compare against base- lines	2,000	38	29K	300	~85% accu- racy on 25%	100% accu- racy on 100%
Deployment correla- tion	Timeline match- ing	6,000	108	54K	930	Moderate accu- racy, 25%	Exact, 100%

Phase	Description	Conv. Tokens	VDR Tokens	VDR GPU Ops	VDR Wall (ms)	Conv. Data Cover- age	VDR Data Cover- age
Statistics + ranking	Aggregate and pri- oritize	2,500	44	5K	440	Errors in arith- metic, 25%	Exact, 100%
Complex trans- form	Python analysis script	700	92	75K	985	Manual by user	Automated sandbox
Versioned storage	Save project state	200	183	20K	300	Nothing saved	Full ver- sioned project
Formatted output	Report + export	1,500	232	86K	2,010	Prose with errors, no export	Grammar tables, CSV + JSON
Total		25,100	769	1.5M	5,715		

Total VDR GPU primitive time: 1.5 ms. Wall clock: 9 seconds versus 660 seconds ($73 \times$ faster). Cost including human time at \$150/hour: \$0.39 versus \$27.58 ($71 \times$ cheaper).

The data coverage column is the capability boundary in action. The conventional LLM can process approximately 50 of 200 Prometheus endpoints before context overflow. VDR processes all 200 through builtins without any data entering the context window.

Appendix Y: Development Time Comparison

LLM Software (VDR-24)

Application	Data Loading	Rule Writing	Testing	Edge Cases	Total	Conventional Equiva- lent
Simple FAQ bot	1 hr	1 hr	1 hr	1 hr	4 hrs	2-4 weeks
Full support chatbot	2 hrs	3 hrs	2 hrs	3 hrs	10 hrs	4-8 weeks
Document processor (single type)	1 hr	2 hrs	1 hr	1 hr	5 hrs	2-4 weeks

Application	Data Loading	Rule Writing	Testing	Edge Cases	Total	Conventional Equivalent
Document processor (5 types)	3 hrs	6 hrs	3 hrs	4 hrs	16 hrs	8-16 weeks
SRE triage assistant	2 hrs	4 hrs	3 hrs	3 hrs	12 hrs	6-12 weeks
Monitoring poller	0.5 hrs	1 hr	0.5 hrs	0.5 hrs	2.5 hrs	1-2 weeks

LLM Server Software (VDR-25)

Service	Grammar Dev	Rule Dev	Testing	Total	Conventional Equivalent
Static HTTP	1 hr	1 hr	1 hr	3 hrs	1-2 weeks
JSON REST API	1 hr	4-8 hrs	2-4 hrs	7-13 hrs	4-8 weeks
DNS authoritative	1 hr	1-2 hrs	1-2 hrs	3-5 hrs	1-2 weeks
MQTT broker	1 hr	2-3 hrs	1-2 hrs	4-6 hrs	2-4 weeks
OAuth/OIDC provider	2 hrs	4-6 hrs	3-4 hrs	9-12 hrs	8-16 weeks
Full email stack	4 hrs	10-16 hrs	4-6 hrs	18-26 hrs	3-6 months
Full monitoring stack	3 hrs	6-10 hrs	3-5 hrs	12-18 hrs	2-4 months

The speedup comes from the same mechanical source in both cases: configuring a session with data, rules, and grammars through conversation rather than writing code, building infrastructure, deploying services, and implementing monitoring from scratch.

Appendix Z: Float Error Classes Structurally Eliminated

Error Class	Conventional Source	Conventional Rate	VDR Mechanism	VDR Rate
Arithmetic	Digit prediction per token	~2-5% per op, compounds	Exact integer builtins	0
State loss	Attention dilution over conversation	~3-5% per turn cumulative	KB persistence at integer addresses	0
Formatting	Structural token prediction	~3-10% per response	Grammar templates	0
Retrieval	Attention over training data	~5% fabrication risk	KB query by integer address	0
Deduction	Reasoning chain in prose	~10% per 5-step chain	Prolog evaluation	0
Confidence	Hedging language, no computation	100% imprecise	Exact VDR fraction from propagation rules	0

These are not statistical improvements. They are categorical eliminations. The error rate is not reduced — the error class does not exist. An exact integer builtin cannot produce a wrong arithmetic result. A KB query by integer address cannot fabricate a fact. A grammar template cannot produce a malformed structure. A Prolog derivation cannot reach an incorrect conclusion from correct premises (it can reach a wrong conclusion from wrong premises — but the error is in the premises, which have inspectable provenance, not in the deduction mechanism).

The remaining error surface is LLM judgment: intent recognition, step selection, assessment of findings, and prose generation. These are the tasks where the LLM’s capability is genuine and irreplaceable. The system focuses the LLM’s compute on exactly these tasks and routes everything else to exact deterministic mechanisms.

Appendix AA: Cumulative Validation Statistics

Paper	Category	Tests	Passed	Failed (Test Error)	Failed (VDR Error)
VDR-1	Core arithmetic	68	68	0	0
VDR-2	15-domain gym	285	279	6	0
VDR-3	8-domain gym + Q335	157	152	5	0

Paper	Category	Tests	Passed	Failed (Test Error)	Failed (VDR Error)
VDR-4	LLM pipeline	198	196	2	0
VDR-12	Grammar compaction	179	178	1	0
VDR-13	14 physics domains	—	—	—	0
VDR-26	Diffusion	37	33	4	0
VDR-27	35 domains	—	—	—	0
Total	38 domains	921	903	18	0

All 18 failures trace to test-design errors: wrong expected values, thresholds too tight, normalization presentation issues, precision frame mismatches, and algorithm implementation bugs in test harnesses. Zero failures from incorrect VDR arithmetic. The system remains falsifiable: any test producing an incorrect exact rational from correct inputs would falsify VDR.

Appendix BB: Datacenter Economics at Scale

Blended $30 \times$ cost reduction across workload mix (conservative — weighted average of $71 \times$ for structured tasks and $2 \times$ for unstructured).

Metric	Conventional	VDR	Reduction
GPU-hours per day (1M requests)	10,000	333	$30 \times$
GPUs required (at utilization)	~420	~14	$30 \times$
Annual GPU cost (\$30K/GPU/yr)	\$12.6M	\$420K	$30 \times$
Annual energy (700W/GPU)	2,570 MWh	86 MWh	$30 \times$
Annual energy cost (\$0.10/kWh)	\$257K	\$8.6K	$30 \times$

These figures do not include engineering cost savings from determinism (reproducible testing, compliance, debugging), structural safety (no ongoing RLHF, red teaming, prompt injection defense), or rule accumulation (decreasing cost over deployment lifetime). Each of these is an additional cost reduction that is real but harder to quantify as a single multiplier.

Appendix CC: Hardware Availability Timeline

Unit	First Available	Platform	Notes
INT8 multiply-accumulate	2017 (Volta V100)	All NVIDIA GPUs since	VDR GEMM path
INT8 tensor cores	2020 (Ampere A100)	A100, H100, B100+	VDR primary compute
INT32 CUDA cores	2006 (Tesla G80)	All NVIDIA GPUs	VDR softmax, activation, KB ops
AVX-512 integer	2017 (Skylake-X)	Intel Xeon	VDR CPU SIMD path
AVX-512 VNNI	2019 (Cascade Lake)	Intel Xeon 2nd gen+	VDR CPU optimized path
ARM NEON integer	2011 (ARMv8)	All modern ARM	VDR ARM path
Apple AMX INT8	2020 (M1)	All Apple Silicon	VDR Apple path
TPU INT8 systolic	2018 (TPU v3)	Google Cloud	VDR TPU path

No hardware development is required. Every compute unit VDR targets has been in production for 6-9 years. The integer throughput trajectory across GPU generations — Volta (2017) through Blackwell (2024) — shows consistent increases in INT8 tensor core throughput and INT32 ALU capacity. VDR rides this trajectory. Float improvements face diminishing returns because the fundamental limitations (non-associativity, special values, transcendental bottleneck) are mathematical properties of the representation, not engineering constraints.

[[HOWL-VDR-33-2026](#)] VDR-LLM-Prolog: The Compound Architecture Performance Gains. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-33-2026>

[[HOWL-VDR-1-2026](#)] VDR Arithmetic: Value, Decimal, Remainder. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-1-2026>

[[HOWL-VDR-2-2026](#)] VDR Gym: Exact Arithmetic Across Fifteen Domains. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-2-2026>

[[HOWL-MATH-3-2026](#)] The Transcendental Hierarchy. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH-3-2026>

[[HOWL-MATH-4-2026](#)] The Universal Power-of-Two Basis. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH-4-2026>

[[HOWL-VDR-14-2026](#)] VDR-LLM-Prolog. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-14-2026>

[[HOWL-VDR-21-2026](#)] VDR-LLM-Prolog on FPGA. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR-VDR-21-2026>

[[HOWL-VDR-22-2026](#)] VDR-LLM-Prolog on Dedicated Silicon. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-22-2026>

[[HOWL-VDR-23-2026](#)] VDR-LLM-Prolog: Functional Remainder Hardware. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-23-2026>

[[HOWL-VDR-24-2026](#)] LLM Software. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-24-2026>

[[HOWL-VDR-25-2026](#)] LLM Server Software. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-25-2026>

[[HOWL-VDR-26-2026](#)] VDR and Diffusion. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-26-2026>

[[HOWL-VDR-27-2026](#)] VDR Beyond Language Models. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-27-2026>

[[HOWL-VDR-28-2026](#)] Exact Rational Arithmetic for Sequential Computation. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-28-2026>

[[HOWL-VDR-29-2026](#)] VDR in Zig SIMD and GPU Performance versus Floating Point. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-29-2026>

[[HOWL-VDR-30-2026](#)] Economics of Scale: Floating Point vs Exact Integer ML Models. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-30-2026>

[[HOWL-VDR-31-2026](#)] VDR Toy LLM. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-31-2026>

[[HOWL-VDR-32-2026](#)] VDR-Zig Q16 Integer LLM. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-32-2026>